

Working together without breaking the code.

Who owns what, how we use GitHub, and what each of us is responsible for.

Sri Charan · On-device AI, systems

Sohan · UI and UX

Prem Sai · Office Kit, content, demo, pitch

Read this once tonight. It takes ten minutes.
It will save hours tomorrow.

TaskLens is built on the open ShowHow project by Kshitij Desai, Anushka Unde and Ayaan Memon. This guide adapts their team guide to our own repo and roles.

The one idea that prevents almost every problem

Git conflicts happen when **two people edit the same file**. Almost nothing else causes them.

So we are not going to be clever about branching strategies. We are going to make sure **no two people ever edit the same file**. If we hold that line, git stays boring, and boring is exactly what you want at three in the morning.

Here is who owns what. If a file is not in your list, you do not open it.

FOLDER OR FILE	OWNER	WHAT LIVES THERE
core/	Sri Charan	Gate, step cutter, mode engine, policy
capture/	Sri Charan	Audio recording, camera
ai/	Sri Charan	Speech, captions, gestures, scene check, Gemma 3 1B coach
data/	Sri Charan	Guide files and storage
ui/	Sohan	Every Compose screen, theme, all visuals
assets/policy.json	Prem Sai	Every tunable number, both word lists
docs/, README.md	Prem Sai	Docs, pitch script, demo notes
build.gradle.kts AndroidManifest.xml	Sri Charan only	Nobody else touches these, ever

THE RULE

If you need something changed in someone else's folder, **ask them**. Do not do it yourself. Ten seconds of asking beats forty minutes of untangling.

Git, for people who have not used it in a team

The five commands you actually need

```
git pull                # get everyone else's latest work
git add .               # stage your changes
git commit -m "what I did" # save them with a message
git push               # send them to GitHub
git status              # what is going on right now
```

That is it. There is no sixth command you need this weekend.

The rhythm

Every time you sit down to work:

```
git pull
```

Every time you finish something that works, roughly every 30 to 60 minutes:

```
git add .
git commit -m "added the mic level meter"
git push
```

Commit small and often. A commit is a save point you can return to. Commit once every six hours and a crash costs you six hours. Commit every thirty minutes and it costs you thirty.

DO NOT BREAK THESE

1. **Never run `git push --force`.** It deletes other people's work. If someone tells you to use it, they are wrong. Only Sri Charan, and only if certain.
2. **Never commit a model file.** They are hundreds of megabytes or gigabytes. They live in `models/` and `.gitignore` keeps them out. If `git status` ever shows a `.task`, `.onnx` or `.tflite` file, stop and tell Sri Charan.
3. **Only push code that builds.** If it does not compile, do not push it to `main`.
4. **Pull before you push.** Always. Every time.

When git says "conflict"

You will see something like:

```
CONFLICT (content): Merge conflict in ui/PlayerScreen.kt
```

Do not panic and do not guess. Say so out loud in the room. If it is a file you own, Sri Charan will walk you through it in two minutes.

The escape hatch, which always works and which you should not be embarrassed to use:

1. Copy your changed file somewhere safe, like your desktop
2. `cd ..` then delete the folder, then clone the repo fresh
3. Paste your file back in
4. Commit and push

It feels crude. It takes four minutes. It has never once made things worse.

Branches, kept minimal

For most of the weekend, work directly on `main`. With file ownership in place this is safe, and it is far faster than pull requests, which none of us have time to review.

Use a branch only when you are trying something risky and might need to throw it away:

```
git checkout -b sohan-new-player-layout # start the experiment
# ...work, commit, push as normal...
git checkout main # go back to safety
```

If the experiment works, tell Sri Charan and the branch gets merged. If it does not, abandon the branch and return to `main`. Nothing is lost.

TONIGHT, ALL THREE OF YOU

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
git clone https://github.com/sricharan5865/iqoo-task-lens
```

Then open it in Android Studio and press Build. **Do this tonight, not tomorrow.** If it does not build on your machine, we need to know now, while there is still time and internet.

Who does what

THE CONSTRAINT THAT SHAPES EVERYTHING

During **Red Light** the laptop is closed as a build machine. That means roughly 10.5 hours where nobody can compile. Our only compile windows are **Saturday 11:00–13:00, Saturday 15:30–16:30, and Sunday 01:00–06:30.**

Everything else is config tuning through `policy.json`, testing on the phone, recording demo guides, and pitch work. Plan accordingly, and confirm the exact rule at the Saturday teach-in.

These windows come from the ShowHow guide. Check them against our own schedule at check-in.

Sri Charan

The spine, everything the product actually does

Realistically, you write most of the Kotlin.

Tonight

Get the repo building under the new package name, get the tests passing, and check the manifest still has no `INTERNET` permission. Send the repo URL to the other two. Start the Gemma 3 1B download before anything else. It is much smaller than the 3n model it replaces, but it still sits behind a Hugging Face licence acceptance.

Saturday Green, 11:00–13:00

Get models onto the phone and loading. This window exists because you can compile in it. Do not spend it on anything you could do during Red Light.

Saturday Green, 15:30–16:30

Your one hour for whatever is genuinely broken. Protect it. Do not fill it in advance.

Sunday Green, 01:00–06:30

Your biggest compile window by far, five and a half hours. All heavy integration goes here. It is also one in the morning until half six, so sleep earlier in the night rather than pushing through the evening.

During Red blocks

You cannot build. Use them to tune `policy.json` values with Prem Sai, test on the phone, and hunt bugs you can fix through config alone.

You also own the technical document and the answers to hard jury questions, including why the coach moved from Gemma 3n E2B to Gemma 3 1B int4. Nobody else can defend the adaptive gate.

Sohan

Everything the judge touches

You own `ui/` completely. Sri Charan will hand you plain, working screens; you make them worth looking at.

Four screens, in priority order

#	SCREEN	WHY IT MATTERS
1	PlayerScreen	The most important screen in the product. Four visual states (TAP, TALK, HANDS, EASY) plus the mode reason bar pinned to the bottom. This is what the jury watches during the demo. TALK needs text readable across a room. HANDS must be usable with no touching at all.
2	ShowScreen	Viewfinder, live mic meter showing level/gate, and the step timeline building up as the expert talks.
3	ReviewScreen	Join, split, re-record. The pitch says a person fixes the cuts in twenty seconds, so this screen has to make that actually true.
4	LibraryScreen	The list of guides. Least important. Do it last.

Tonight

Clone the repo, confirm it builds, and spend an hour on Compose basics if it is new to you. Focus on `Column`, `Row`, `Box`, `Text`, `Button`, `Modifier` and `remember`. That is genuinely enough to build all four screens.

Carry the deck into the app

Pick the visual language once, paper background, black display type, blue accents, and use it in both the slides and the app, so they look like the same thing. Since the rebrand, check that the app name and icon say TaskLens everywhere on screen.

During Red blocks

You cannot compile either, but you can design. Sketch layouts, pick type scales and colours, and have them ready to type the moment a Green window opens.

Prem Sai

The two scoring categories nobody else has time for

READ THIS PART FIRST

Two categories, 20% of the total, are yours alone. **Office Kit usage is 10%**, read off HackTracker device data, real counts and durations, not self-reporting, so it cannot be faked on Sunday morning. **Demo and presentation is another 10%**, judged by humans on a three to five minute pitch.

You also materially affect a third. **End product quality is 30%**, judged on a two-and-a-half minute live demo. If the recording fumbles on stage, that 30% goes with it. The five guides you record on Sunday morning are the insurance.

Sri Charan and Sohan will be writing code until the last hour. Neither of them will have time for any of this.

The five things you hand over

Everything you do produces one of these. If it is not on this list, it is not your job.

#	DELIVERABLE	WHERE IT LIVES
1	Every tunable number, tuned	assets/policy.json
2	Hindi and Marathi linking-word lists	inside policy.json
3	The Office Kit usage log	docs/officekit-log.md
4	Demo runbook + five recorded guides	docs/demo-runbook.md
5	Pitch script + the ~100 second video	docs/pitch.md

Sri Charan writes the code. Sohan makes it look right. You make sure there is something to show, and someone who can explain it.

Tonight

1. Install Office Kit on your laptop. `pc.vivoglobal.com`. Windows 10 or later, macOS 10.14.6 or later. It is already on the loaner phone, so this is the only half you need.
2. Clone the repo and run the setup check. If your machine cannot build the project, we need to know tonight, while there is still internet.
3. Read the project context document twice. Not skim, read. Tomorrow you need to explain the adaptive gate to a stranger who is judging you. If you cannot yet, ask Sri Charan until you can.
4. Draft the word lists. Start from *phir*, *ab*, *uske baad*, *next*, *then*. Add every other way you have heard someone move to the next step while explaining something. Aim for fifteen to twenty in Hindi and ten in Marathi. Thirty minutes, and it is the input to half the product. We are in Hyderabad, so if Sri Charan confirms the speech model handles Telugu, note down Telugu step words too, such as *tarvata*.
5. Write a first pass of the pitch. It will be wrong. Write it anyway. Editing something bad on Saturday is far easier than starting from blank.

Prem Sai • your hour by hour

Saturday

WHEN	LIGHT	WHAT YOU DO
10:00–11:00	CHECK-IN	You collect the phone. Three jobs, all yours, see the box below.
11:00–13:00	GREEN	Sri Charan is loading models onto the phone. Move every one of those files by Office Kit file transfer, not by cable. Same result, and it starts your usage clock in hour one. Log each transfer.
13:00–15:30	RED	Expand the word lists. Walk around and ask people how they explain a process out loud, you are at a venue full of them. Start <code>docs/demo-runbook.md</code> as numbered steps someone else could follow.
15:30–16:30	GREEN	Sri Charan's emergency hour. Stay out of the way. Keep screen mirror running so you can both see the phone, and be there to fetch, hold, time and read errors aloud.
16:30–19:00	RED	Your main working block. Tuning <code>policy.json</code> with Sri Charan. Config is the only thing anyone can change during Red Light and you are the one with hands on the file. Every change goes over by file transfer. Log every one.
19:00–22:00	EVAL 1	This round is scored. You present. Use the draft pitch and find out what confuses people, that is the whole value of this round. Write down every question you could not answer well and hand the list to Sri Charan.
22:00–00:00	RED	Test guide playback end to end. Record throwaway practice guides and learn what makes a bad one, before it matters.

AT CHECK-IN • THREE THINGS ONLY YOU DO

1. Enable developer options and USB debugging. On iQOO there is a second toggle, "USB debugging (security settings)", which usually needs a vivo account sign-in. Do it while you still have network. Skip it and nobody can install anything on that phone all weekend.
2. Pair Office Kit and confirm all four functions work: screen mirror, shared clipboard, file transfer, remote control.
3. Ask the organisers, out loud, one question: **during Red Light, can the laptop compile and deploy at all, or only run Office Kit?** Our whole schedule depends on the answer. Get it from a person, not the website.

Sunday

WHEN	LIGHT	WHAT YOU DO
00:00–01:00	RED	Wrap up, hand your notes to Sri Charan, then go to bed.
01:00–06:30	GREEN	Sleep. This is not a suggestion. It is the only long window in the event, the other two are working through it, and you are the one who has to be coherent at noon. A tired pitch costs more than a tired coder.
06:30–09:00	RED	The block that protects 30% of the score. Record five real guides, properly, start to finish. Run the full demo in airplane mode at least three times and time it, it must fit in two and a half minutes. Break it on purpose: cover the mic, move the phone mid-step, make the room loud. Shoot the ~100 second video.
09:00–12:00	EVAL 2	Round two and the final pitch. You present. Say it out loud twenty times before this, not five.

The pitch, and the log

How to structure the pitch

Same shape, every time you give it.

1. **The problem, in one sentence.** Someone in every workshop knows how the job is done. Writing it down takes hours nobody has.
2. **Show Mode.** The expert does the job once and talks through it. Ninety seconds later it is a guide, in their own recorded voice.
3. **Guide Mode.** The next person runs it with full hands. The phone reads the room and picks how to help. They never choose a mode.
4. **The honest part.** Say plainly what we refuse to claim: gloves, driving, whether someone is elderly, and whether the work is correct. Anything the phone cannot honestly measure is a setting, not a guess.
5. **Where it came from.** One sentence: TaskLens is built on the open ShowHow project, and our contribution is the coach-model swap from Gemma 3n E2B to Gemma 3 1B int4, plus the rebrand. Crediting the original team out loud costs nothing and pre-empts the hardest question.
6. **The proof.** No `INTERNET` permission, and you can show it in the app info screen in four seconds.

DO NOT CUT POINT FOUR OR POINT FIVE

Point four is the one most teams do not have. A team that states its limits reads as more credible, not less, and technical judges notice immediately. Point five keeps us honest about what we built ourselves.

Keeping the Office Kit log

Plain text, `docs/officekit-log.md`, one line each time:

```
Sat 11:20 file transfer gesture_recognizer.task -> phone
Sat 11:34 file transfer gemma3-1b-it-int4.task -> phone
Sat 13:05 screen mirror on, 40 min, testing step cutter
Sat 17:12 file transfer policy.json v4 -> phone
```

Two reasons. It is your evidence for the 10%, and it feeds straight into the technical document. Steady use across all 30 hours scores better than a burst at the end, so make it a reflex from hour one.

When you are not blocked and nobody needs you

Hold the phone. Run the tests. Time things. Read error messages aloud so Sri Charan does not have to look away from the keyboard. Take notes while the other two debug.

There is no such thing as waiting on a three-person team.

Rhythms for the weekend

Fifteen minutes before every Green window opens

Decide out loud what gets compiled in it. Green time is the scarce resource. Do not discover at 15:45 that you spent the hour on something you could have done during Red Light.

Every four hours, one person says out loud what the demo would look like if we stopped right now

If the answer is "nothing works," we cut scope immediately, rather than at hour 28.

Whoever is not blocked helps whoever is

Prem Sai can hold the phone, run tests, time things, read errors aloud. Sohan can take notes during debugging. There is no such thing as waiting.

Sleep in shifts

Nobody stays up 30 hours and pitches well at noon on Sunday. Prem Sai is pitching, so Prem Sai sleeps most.

THE SHORT VERSION

Only touch your own folders. Pull before you push. Commit every thirty minutes. Never force push. Never commit a model file. If git scares you, say so out loud instead of guessing.